



Sharif University of Technology
Department of Mechanical Engineering

Measurement and Control Systems Course Project Report

Title
**Design and Construction of a Step Counter Using Arduino and the
MPU6050 Accelerometer**

Prepared by:
Nima Nouri

Course Instructor
Dr. Nejat

Summer 2026

Abstract

In this project, a step-counting system based on the MPU6050 sensor and an Arduino Uno board was designed and implemented. The Arduino acquires the three acceleration components at a sampling rate of 50 Hz. The acceleration-vector magnitude is calculated, the slowly varying baseline is removed, the signal is smoothed, local peaks are detected, and an adaptive threshold is applied. The step count is displayed in real time on an LCD, while raw and processed data are stored on a memory card in CSV format. In the nine pilot trials reported here, 180 manually counted steps were compared with 178 detected steps, corresponding to an overall count agreement of 98.9% and a mean absolute error of 0.22 steps per trial. Because each speed-location condition was recorded only once, these pilot results do not establish repeatability; a repeated-trial protocol is therefore specified in the evaluation section.

Table of Contents

Abstract.....	2
Introduction and Project Objectives.....	4
Hardware Design and Connections.....	6
Signal Processing and Step Detection Method	7
Sampling and Acceleration-Magnitude Calculation.....	7
Baseline Removal	8
Signal Smoothing.....	8
Adaptive Threshold.....	8
Peak Detection and Step Counting.....	9
Data Display and Storage.....	10
LCD Display	10
Evaluation of the Algorithm and Adaptive Threshold at Different Sensor Locations	11
Data Recording on the Memory Card.....	13
Setup Testing Procedure	13
Results and Error Calculation.....	14
Discussion and Error Analysis.....	16
Conclusion	17
References	18

List of Figures

Figure 1 Project components used in the prototype.....	
Figure 2 Detailed wiring schematic of the step-counter prototype.....	
Figure 3 Flowchart of the signal-processing and step-detection algorithm	
Figure 4 Step-count display on the LCD	
Figure 5 Dynamic acceleration and adaptive threshold for different sensor locations	
Figure 6 Prototype mounted on the hand chest and leg	
Figure 7 Acceleration-signal processing stages during walking.....	

Introduction and Project Objectives

A step counter is a common application of inertial sensors in wearable systems and physical-activity monitoring. During walking, periodic body motion produces distinct patterns in the acceleration signal. By mounting the sensor on a suitable part of the body and analyzing these variations, each valid movement cycle can be identified as a step. However, in addition to the motion caused by walking, actual sensor data contain measurement noise, the gravitational component, changes in sensor position and orientation, hand motion, and unwanted vibrations. Therefore, direct use of the raw data generally leads to incorrect counting, and the sensor signal must be processed and filtered before step detection.

The objective of this project is to design and implement a practical, stand-alone step-counting system that not only counts steps in real time but also enables the measured data to be viewed and stored. The expected outputs include live display of the step count, recording of the acceleration signal during walking, comparison of the actual and detected step counts, calculation of counting error, and investigation of the effects of factors such as sensor location, walking speed, signal noise, additional body movements, and threshold adjustment on system performance.

The required components are listed below.

No.	Component	Function
1	Arduino Uno	Data processing and algorithm execution
2	MPU6050	Three-axis acceleration measurement
3	Jumper wires and breadboard	Connecting the components
4	LCD 16×2	Displaying the step count
5	Memory-card module	Recording data in CSV format
6	4 GB SD card and card reader	Data storage
7	Retaining strap or elastic band	Securing the setup on the body
8	5 V boost converter	Converting the battery voltage from 3.7 V to 5 V
9	Lithium battery and battery holder	Power supply
10	100-ohm resistor	Limiting LCD-backlight current, if required by the module

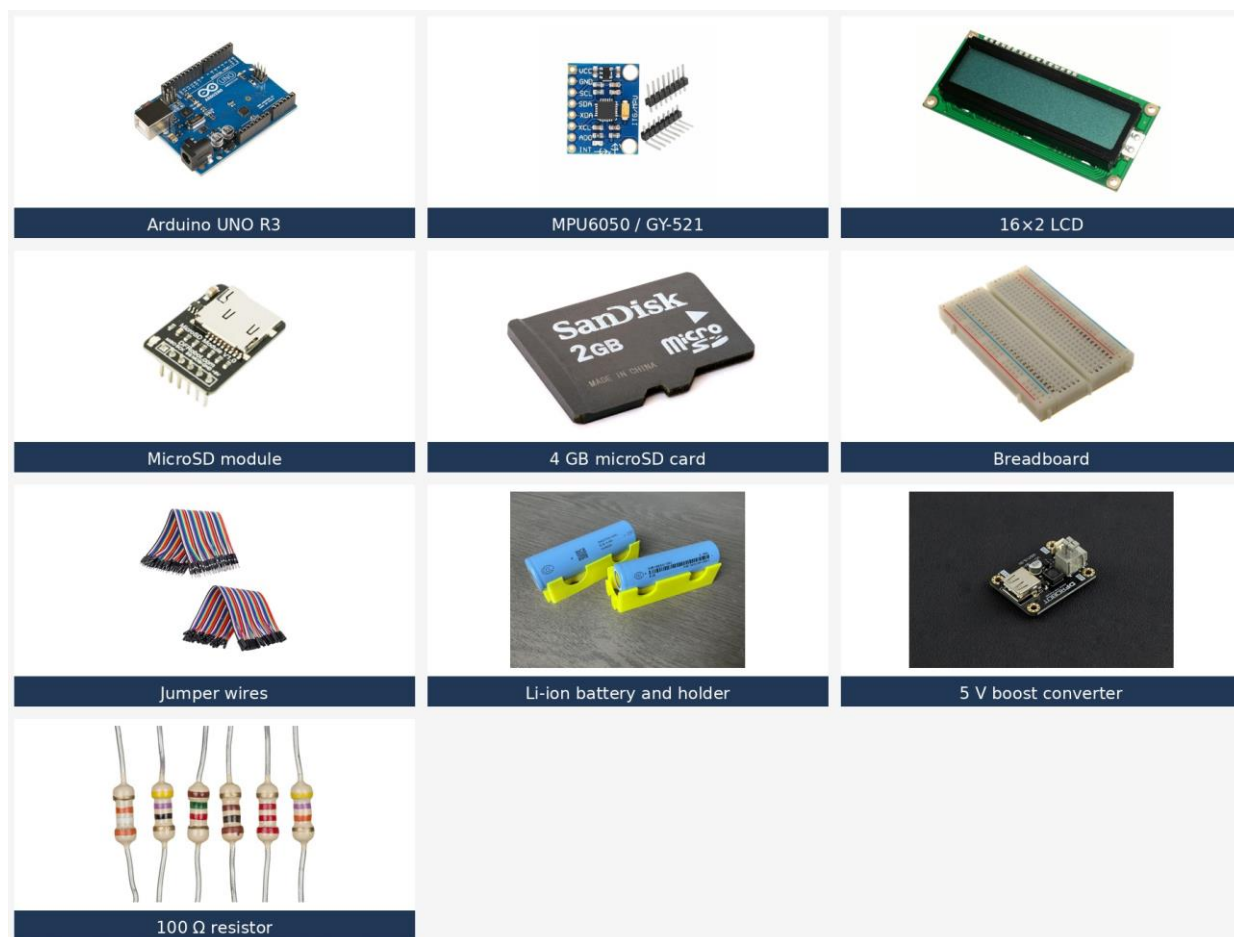


Figure 1. Project components used in the prototype (representative product images; the exact breakout-board appearance may vary).

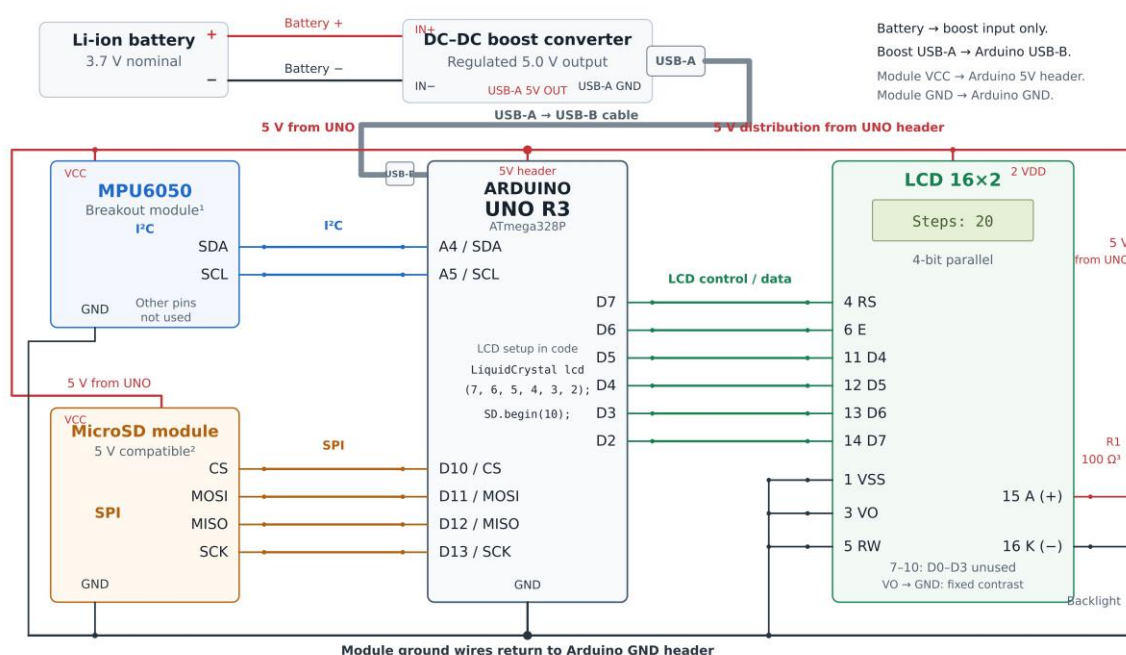
Hardware Design and Connections

The MPU6050 sensor is connected to the Arduino board through the I²C communication bus. The LCD is operated in 4-bit mode to reduce the number of required Arduino pins. The MicroSD module uses the SPI interface for data transfer and storage. The table lists the signal connections; the exact VCC requirement of the MicroSD breakout must be checked because some modules include level shifting and regulation whereas a bare SD card is a 3.3 V device. All grounds are connected to a common Arduino GND. The battery powers the boost converter, whose USB-A output is connected by a USB-A-to-USB-B cable to the Arduino USB-B port. The modules take power from the Arduino 5V pin. In the prototype, LCD pin VO is tied to GND, so the contrast is fixed; an approximately 10-k Ω potentiometer between 5 V and GND, with its wiper connected to VO, is the appropriate adjustable-contrast arrangement. The 100-ohm resistor is associated with the LCD backlight current path, not with contrast adjustment.

MPU6050 Step Counter

Pin-by-pin wiring • Arduino UNO R3 • LCD 16x2 in 4-bit mode

PROJECT WIRING



1 / 2 Boost USB-A output connects to Arduino USB-B; the modules are powered from the Arduino 5V header.
Bare MPU6050 / SD devices use 3.3 V circuitry; use compatible breakouts or add regulation and level conversion.
3 R1 follows the report. Confirm the LCD backlight current and any resistor already on the module.
Optional contrast: replace VO → GND with a 10 kΩ potentiometer; end to +5V / GND, wiper to VO.

Figure 2. Detailed wiring schematic of the step-counter prototype.

Module	Module Pin	Arduino Pin
MPU6050	VCC	5V
MPU6050	GND	GND
MPU6050	SDA	A4
MPU6050	SCL	A5
MicroSD	CS	D10
MicroSD	MOSI	D11
MicroSD	MISO	D12
MicroSD	SCK	D13
LCD	RS	D7
LCD	E	D6
LCD	D4	D5
LCD	D5	D4
LCD	D6	D3
LCD	D7	D2
LCD	RW	GND
LCD	VSS	GND
LCD	VDD	5V
LCD	VO	GND

Power note: verify the exact VCC and level-shifting arrangement of the MicroSD breakout before connecting it to a 5 V Arduino. The boost USB-A output connects to the Arduino USB-B port. The other modules draw power from Arduino 5V, and their GND wires terminate at Arduino GND.

Signal Processing and Step Detection Method

Sampling and Acceleration-Magnitude Calculation

Three-axis acceleration is read with a sampling period of 20 ms; therefore, the sampling rate is 50 Hz. To reduce the dependence of the algorithm on the sensor mounting orientation, the magnitude of the acceleration vector is calculated from the following equation instead of using a single fixed axis.

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad , \quad f = \frac{1000}{20} = 50 \text{ Hz}$$

Baseline Removal

At rest, the acceleration-vector magnitude consists approximately of gravitational acceleration and accelerometer bias. To remove these components as well as slow variations in the signal, a 50-sample moving average is calculated as the baseline. The difference between the acceleration-vector magnitude and the baseline is then considered the dynamic acceleration. At a sampling rate of 50 Hz, the 50-sample window covers approximately one second of data.

$$a_{dyn} = a_{magnitude} - a_{baseline}$$

baseline= moving average of the latest 50 magnitude samples

Signal Smoothing

To reduce noise and rapid fluctuations in the signal, a moving-average low-pass filter is used. In this method, a specified number of the most recent dynamic-acceleration samples are considered at each instant, and their average is calculated as the filter output. For example, with a 7-sample window, the output at each instant is the average of the latest 7 samples. When a new sample is received, the calculation window advances by one step: the new sample enters the calculation, and the oldest sample is no longer included in the new average.

This process reduces the influence of sudden, short-duration variations that are generally caused by sensor noise or very rapid vibrations. Because a sudden spike is only one of several samples included in the average, its effect is distributed among the other samples and its amplitude is reduced. In contrast, the main continuous variations in the signal that persist over several consecutive samples are retained in the output. Consequently, the moving-average filter attenuates high-frequency components and produces a smoother and more readily detectable signal pattern associated with movement and steps.

Adaptive Threshold¹

The amplitudes of acceleration peaks are not constant for different individuals, sensor mounting locations, and walking speeds; therefore, an adaptive threshold is used. First, the local peaks of the signal are identified. Any peak with a height of at least 0.5 is considered a valid peak for threshold adjustment. The average height of these peaks is updated using an exponential average, in which the new peak has a weight of 0.4 and the previous average has a weight of 0.6. This method does not use a fixed number of peaks, and the influence of

¹ Adaptive threshold

older peaks gradually decreases; therefore, the most recent few peaks have the greatest effect. The step-detection threshold is then set to a percentage of this average, and a minimum threshold value of 0.5 is used to prevent it from decreasing excessively.

$$\text{averagePeak}_{\text{new}} = (1 - \alpha) * \text{averagePeak}_{\text{old}} + \alpha * \text{peak}$$

$$\text{Threshold} = 0.55 \times \text{averagePeak}$$

$$0.60 \leq \text{Threshold} , \alpha = 0.40$$

Peak Detection and Step Counting

To detect a peak, each set of three consecutive samples is compared. The middle sample is considered a local peak when its value is greater than the previous sample and greater than or equal to the following sample. However, not every local peak necessarily represents a step. A peak is counted as a step only when its height exceeds the adaptive threshold and at least 400 ms have elapsed since the previous step was detected. This time-interval condition prevents several closely spaced fluctuations or peaks produced by a single movement from being incorrectly counted as separate steps.

$$\text{LocalPeak} : A_{k-1} > A_{k-2} \text{ and } A_{k-1} \geq A_k$$

$$\text{validStep: localPeak and } A_{k-1} > \text{Threshold} \text{ and } \Delta t \geq 400 \text{ ms}$$

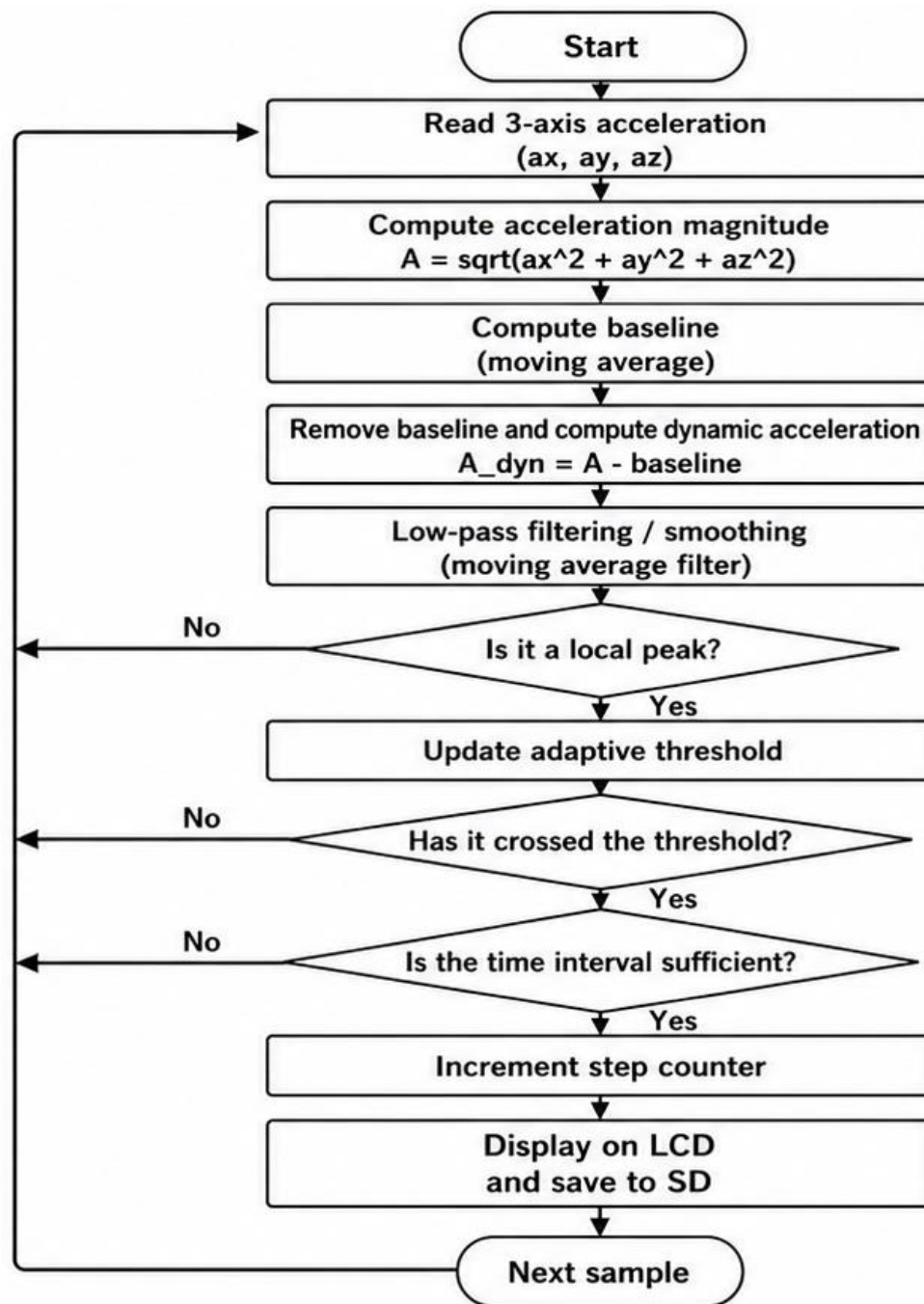


Figure 3. Flowchart of the signal-processing and step-detection algorithm.

Data Display and Storage

LCD Display

After each valid step is detected, the counter is incremented by one and its new value is shown on the LCD. To reduce delay and prevent visible jumping or flickering, only the portion of the display containing the counter value is updated at each stage, and the entire display is not cleared.

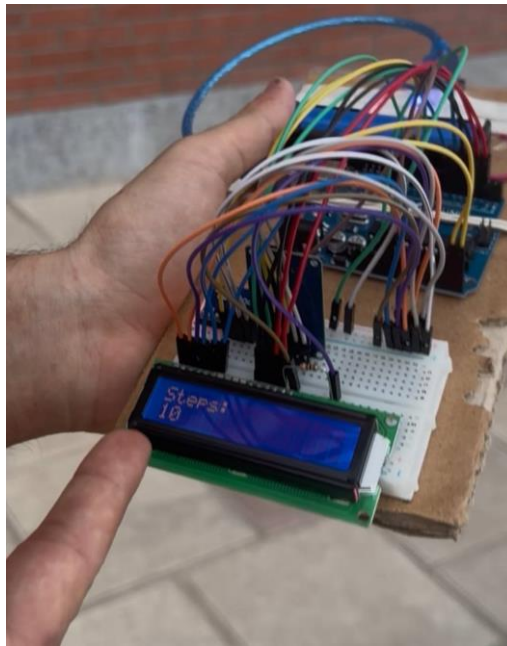


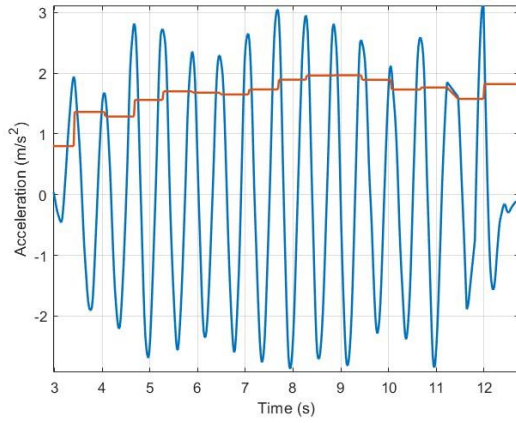
Figure 4. Step-count display on the LCD.

Evaluation of the Algorithm and Adaptive Threshold at Different Sensor Locations

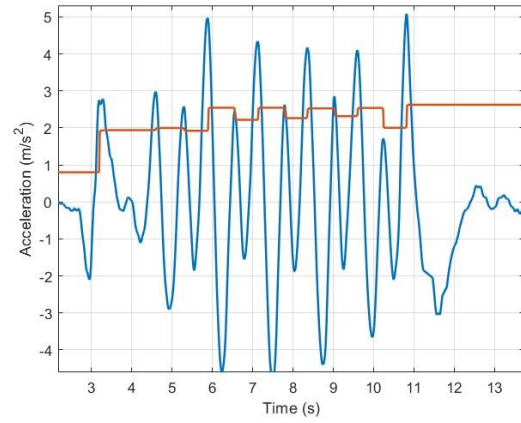
To investigate the effect of sensor location on algorithm performance, the sensor-and-circuit assembly was tested in several conditions, including being held in the hand, mounted on the chest, and mounted on the leg. In the plots, the blue curve represents the smoothed dynamic acceleration, and the orange curve represents the adaptive threshold. The results showed that the amplitude and regularity of the acceleration peaks changed with sensor location; nevertheless, the adaptive threshold varied according to the height of the signal peaks and adjusted itself to the conditions of each test.

When the sensor was mounted securely on the chest, all three pilot counts matched the manual count. For the leg condition, two pilot counts matched and the fast-walking trial undercounted by one step. Secure mounting reduced additional movements unrelated to walking and generally produced a clearer relationship between signal peaks and actual steps.

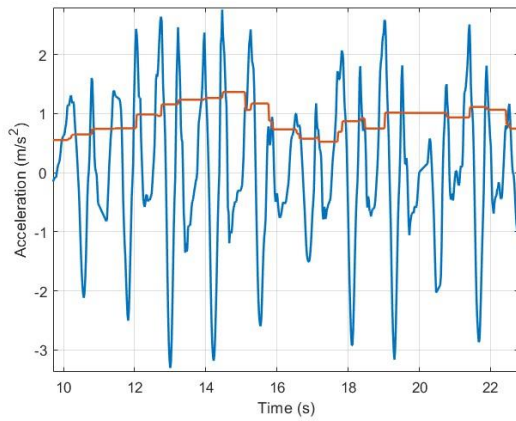
When the sensor-and-circuit assembly was held in the hand, sudden hand movements could generate peaks similar to step peaks. Such peaks are a potential source of false counts; in the pilot table, the slow-hand trial produced an undercount of one step and no overcount was recorded. These results show that the adaptive threshold responds to signal amplitude, but it cannot identify the physical source of a movement.



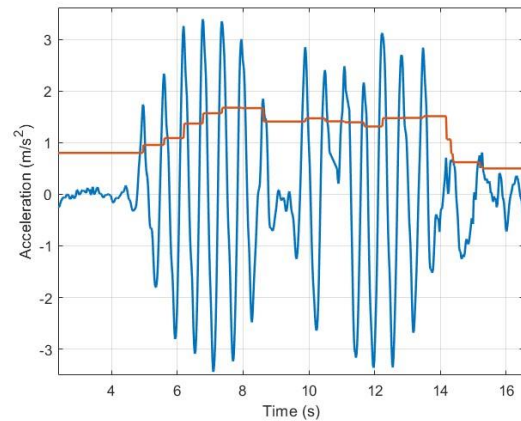
Panel 1



Panel 2



Panel 3



Panel 4

Figure 5. Dynamic acceleration and adaptive threshold for different sensor locations.

In Figure 5, panels 1 and 2 correspond to the hand-held condition. In panel 1, the assembly was held carefully with minimal additional movement; in panel 2, it was held normally and natural hand motion was more prominent.

In Figure 5, panels 3 and 4 correspond to the leg-mounted and chest-mounted conditions, respectively. Because the sensor was secured to the body, fewer unrelated movements entered the signal and the peaks had a more regular relationship with the walking steps.

Data Recording on the Memory Card

The data from each test are stored in the data.csv file. To prevent data from different tests from being combined, the previous file is deleted and a new file is created each time the Arduino is powered on or reset. The file contains the sample-recording time, acceleration-vector magnitude, dynamic acceleration, filtered acceleration, adaptive-threshold value, step-detection status, and total number of counted steps.

Column	Description
time_ms	Elapsed time since program start, in milliseconds
magnitude	Raw acceleration-vector magnitude
A_dyn	Acceleration after baseline removal
A_f	Smoothed acceleration used for detection
threshold	Adaptive-threshold value
step	Zero or one to indicate a step event in the same sample
stepCount	Cumulative number of steps

Setup Testing Procedure

To evaluate count agreement, the actual number of steps is counted manually and compared with the value displayed on the LCD and the value recorded in data.csv. The table in this report is a pilot dataset: it contains one 20-step run for each speed-location condition. A repeatability claim requires independent repeated trials; the recommended protocol is to repeat every condition at least five times after resetting the device, while keeping the route, step target, mounting position, and walking-speed instruction fixed.

The steps of each test are as follows:

1. The sensor is placed at the selected location on the body and secured with a suitable strap or adhesive.
2. The device is powered on, and proper operation of the sensor, LCD, and memory card is verified.
3. The participant walks a specified number of steps, such as 20, 50, or 100, at the selected speed.
4. The sensor location, walking speed, and actual number of steps are recorded.
5. The detected step count is obtained from the LCD and the data.csv file.
6. The counting error is calculated, and the test is repeated for the other conditions and sensor locations.



Figure 6. Prototype mounted on the hand, chest, and leg (left to right).

Results and Error Calculation

Pilot test results (one run per speed-location condition; 20 manual steps per run)

Test	Speed	Mounting Location	Actual Steps	Detected Steps	Absolute Error	Error Percentage
1	Slow	Chest	20	20	0	0
2	Normal	Chest	20	20	0	0
3	Fast	Chest	20	20	0	0
4	Slow	Hand	20	19	1	%5
5	Normal	Hand	20	20	0	0
6	Fast	Hand	20	20	0	0
7	Slow	Leg	20	20	0	0
8	Normal	Leg	20	20	0	0
9	Fast	Leg	20	19	1	%5

$$\text{Absolute Error} = |\text{Detected Steps} - \text{Actual Steps}|$$

$$\text{Percentage Error} = (\text{Absolute Error} / \text{Actual Steps}) \times 100$$

Pilot accuracy and repeatability assessment

Across the nine pilot runs, the reference total was 180 steps and the detected total was 178 steps. The total absolute error was 2 steps. Overall count agreement was $178/180 = 98.9\%$; mean absolute error was $2/9 = 0.22$ steps per run; mean absolute percentage error was 1.11% ; the exact-match rate was $7/9 = 77.8\%$; and the signed bias was -0.22 steps per run, indicating a small undercount in this sample. The largest observed absolute error was one step.

These numbers describe agreement of the final counts only. They do not provide step-level sensitivity, precision, false-positive rate, or repeatability because each condition was observed once and the event times were not independently labelled.

For a repeatability test, perform at least five independent runs for each location and speed. Before resetting, copy the previous data.csv file to a uniquely named archive because the current program overwrites it; then reset the device for the next run; use a fixed target such as 20 or 50 steps; record the manual count, LCD count, CSV count, location, speed, and any missed or extra events. Report the mean detected count, standard deviation, coefficient of variation (standard deviation divided by the mean), range, mean absolute error, and a 95% confidence interval. The present report therefore labels repeatability as not yet established rather than inferring it from the nine single runs.

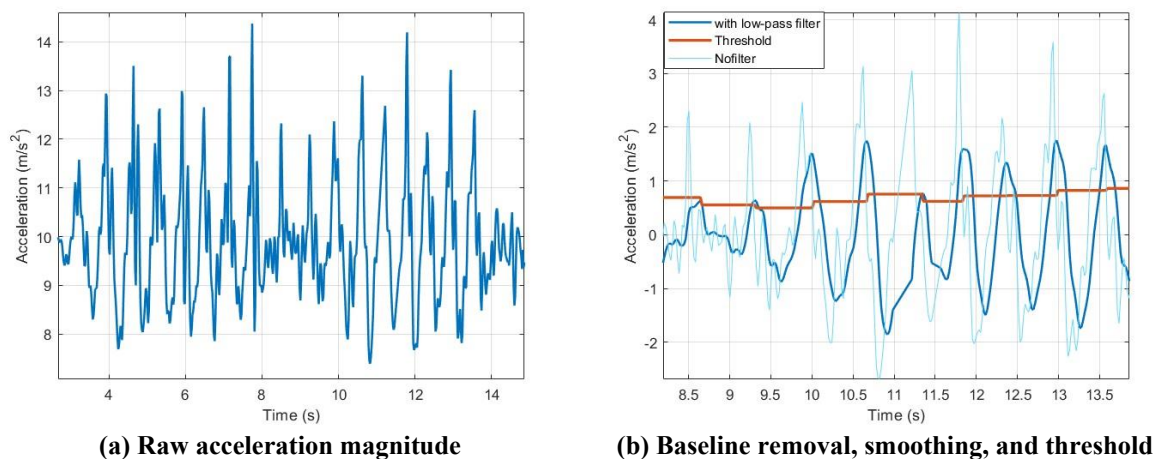


Figure 7. Acceleration-signal processing stages during walking.

Panel (a) - Raw acceleration-vector magnitude recorded by the MPU6050

Figure 7(a) shows the raw acceleration-vector magnitude. Because the gravitational-acceleration component has not yet been removed from the signal, the acceleration value is mainly around 9.81 m/s^2 . The observed fluctuations and peaks are caused by body motion and the impacts produced during walking.

Panel (b) - Comparison of baseline-removed acceleration, filtered acceleration, and adaptive threshold

In Figure 7(b), the light-blue curve represents the acceleration after baseline removal and before filtering. This signal fluctuates around zero but still contains noise and sudden peaks. The dark-blue curve is the output of the moving-average low-pass filter, which reduces rapid fluctuations and makes the step-related peaks clearer. The orange curve represents the

adaptive threshold, which changes according to the amplitude of the signal peaks and is compared with the filtered acceleration for step detection.

Discussion and Error Analysis

The most important source of error is the similarity between some hand or body movements and the acceleration pattern produced during walking. For example, when the sensor is held in the hand, sudden hand movements may generate peaks higher than the adaptive threshold and may incorrectly be counted as steps. In the present algorithm, peaks below 0.5 m/s^2 are not included in the adaptive-threshold calculation, and the minimum threshold value is set to 0.5 m/s^2 . These conditions eliminate many small vibrations, but strong hand movements can still cause false counts.

Sensor mounting location: In the pilot data, chest mounting gave three exact counts; hand and leg mounting each gave two exact counts and one one-step undercount. This small dataset cannot establish a general ranking of mounting locations. Secure mounting may reduce unrelated motion, which should be checked in repeated trials.

Threshold adjustment: A threshold that is too low causes vibrations and weak movements to be identified as steps. A threshold that is too high may cause slow steps and low-amplitude peaks to be missed. The use of an adaptive threshold reduces this problem because the threshold value changes according to the height of recent peaks.

Walking speed: As the walking speed changes, the peak amplitudes and the time intervals between them also change. During slow walking, the peaks are generally weaker and farther apart, whereas during fast walking they are larger and closer together. Therefore, the threshold value and the minimum time interval between two steps affect algorithm performance.

Smoothing-window size: The window is the number of samples used to calculate each output value of the moving-average filter. In this project, a 7-sample window is used; therefore, each filtered-acceleration value is calculated from the average of the latest 7 samples. At a sampling rate of 50 Hz, seven samples represent 0.14 s of sampling time (0.12 s between the first and last sample). Increasing the window size reduces noise and rapid fluctuations more effectively, but it also decreases the peak amplitudes and increases the detection delay. A smaller window provides a faster response and preserves the peaks more effectively, but allows more noise to pass through the filter. The 7-sample window is a design choice; a comparison with other window lengths is needed to justify its performance.

Conclusion

In this project, a stand-alone step counter was constructed using an Arduino Uno board, an MPU6050 sensor, an LCD, and a memory card. The three-axis acceleration was read at a sampling rate of 50 Hz, and the acceleration-vector magnitude was calculated from the x, y, and z components to reduce the dependence of the algorithm on sensor orientation. A 50-sample moving average was then used to reduce the baseline and the constant and slowly varying components of the signal. In the next stage, a 7-sample moving-average filter was applied to reduce noise and smooth the signal so that step-related peaks became clearer.

Step detection was performed by identifying local peaks in the filtered signal. The threshold changes adaptively in proportion to recent peak amplitudes. In the method described here, the minimum threshold is 0.5 m/s^2 , peaks at or above 0.5 m/s^2 contribute to the adaptive estimate, and a minimum 400 ms interval between detected steps prevents closely spaced fluctuations from being counted separately.

The pilot results showed exact count agreement for all three chest trials, two exact counts for the leg condition, and one-step undercounts in the fast-leg and slow-hand trials. No overcount appears in the nine-row pilot table. The pilot observations suggest that stable mounting may help, but they are not enough to establish repeatability or general performance across users and walking conditions.

The real-time display of the step count on the LCD and the simultaneous storage of raw acceleration, acceleration after baseline removal, filtered acceleration, threshold value, and step-detection status in the data.csv file made it possible to evaluate algorithm performance and analyze the data in MATLAB or Excel. The nine pilot trials demonstrate the prototype under the reported conditions; performance in longer walks, across different participants, and during non-walking activities remains to be tested. To improve accuracy under more complex conditions, future versions can include features such as step-timing regularity, peak width, gyroscope variations, and a combination of several signal features in addition to peak amplitude.

References

[1] InvenSense Inc., MPU-6000 and MPU-6050 Product Specification, Revision 3.1, October 2011.

[2] Arduino, Arduino UNO R3 Documentation, Arduino Documentation, accessed July 2026.

[3] Adafruit Industries, MPU6050 6-DoF Accelerometer and Gyro: Arduino Guide, accessed July 2026.

[4] N. Zhao, “Full-Featured Pedometer Design Realized with 3-Axis Digital Accelerometer,” Analog Dialogue, Vol. 44, No. 6, June 2010.

[5] Arduino, SD Library for Arduino, Arduino Libraries Documentation, accessed July 2026.

[6] Arduino, LiquidCrystal Library, Arduino Libraries Documentation, accessed July 2026.

[7] Arduino, “UNO R3,” official hardware documentation, <https://docs.arduino.cc/hardware/uno-rev3> (accessed October 2026).

[8] Adafruit Learning System, “MPU6050 6-DoF Accelerometer and Gyro: Arduino,” <https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro/arduino> (accessed October 2026).

[9] DFRobot, “Fermion MicroSD Card Module for Arduino,” <https://www.dfrobot.com/product-875.html> (accessed October 2026).

[10] DFRobot, “DC-DC Boost Module 0.9–5 V to 5 V,” <https://www.dfrobot.com/product-1475.html> (accessed October 2026).

[11] Wikimedia Commons, “Arduino Uno – R3,” https://commons.wikimedia.org/wiki/File:Arduino_Uno_-_R3.jpg (accessed October 2026).

[12] Wikimedia Commons, “16x2 LCD Display,” [https://commons.wikimedia.org/wiki/File:16x2_LCD_Display_\(cropped\).jpg](https://commons.wikimedia.org/wiki/File:16x2_LCD_Display_(cropped).jpg) (accessed October 2026).

[13] Wikimedia Commons, “400 points breadboard,” https://commons.wikimedia.org/wiki/File:400_points_breadboard.jpg (accessed October 2026).

[14] REES52, “GY-521 MPU6050 Gyroscope Module,” <https://rees52.com/products/gy-521-mpu6050-gyroscope-module-accel-gyr0-3-axis-gyro-sensors> (accessed October 2026).

[15] REES52, “DuPont Jumper Wires Pack,” <https://rees52.com/collections/dupont-jumper-wires> (accessed October 2026).

[16] Wikimedia Commons, “MicroSD card 2GB focus-stacked,” https://commons.wikimedia.org/wiki/File:MicroSD_card_2GB_focus-stacked.jpg (accessed October 2026).

[17] Wikimedia Commons, “18650 and 21700 lithium-ion battery cell,” https://commons.wikimedia.org/wiki/File:18650_and_21700_lithium_ion_battery_cell.jpg (accessed October 2026).

[18] Wikimedia Commons, “Electronic axial-lead resistors array,” <https://commons.wikimedia.org/wiki/File:Electronic-Axial-Lead-Resistors-Array.jpg> (accessed October 2026).